

Dataflow Optimization within High-Level Synthesis

Milestone 2: Scientific Contextualization and Tradeoff Analysis

Electronics Engineering
Hochschule Hamm-Lippstadt

Where This Work Sits in the HLS Landscape

- HLS research can be divided into two broad lines:
 - **Scheduling algorithms** – how to assign operations to clock cycles (list scheduling, ILP, force-directed scheduling, simulated annealing, etc.).
 - **Resource optimization** – how to allocate, share, and select hardware units to minimize area or power.
- The seed paper [1] sits at the intersection of both lines: it performs scheduling, module selection, resource sharing, and power/area optimization *simultaneously* inside a hierarchical iterative improvement framework.
- Most prior work addressed these problems *separately* or only for flat (non-hierarchical) designs. The contribution of [1] is to combine them for hierarchical DFGs.

Related Approaches and Competing Methods

Scheduling Approaches

- **List-based scheduling** [4]: assigns operations greedily by priority (e.g., critical path length). Fast but does not optimize power or area directly.
- **ILP-based scheduling**: formulates scheduling as an integer linear program. Optimal but exponential runtime; not practical for large designs.
- **Force-directed scheduling**: balances resource usage across clock cycles. Better quality than list scheduling but still does not consider power.
- **Simulated annealing** [5]: used in [1] as inspiration for the variable-depth iterative improvement; can escape local minima but is slow to converge.

Power Optimization Approaches

- **Transformational approach** [3]: applies algebraic transformations to the DFG (retiming, associativity, commutativity) to reduce switching activity. Works on flat DFGs only; does not handle hierarchy.
- **Operand isolation**: inserts gating logic to prevent unnecessary switching when a unit's output is not needed. Complementary to the seed paper's approach; not integrated into [1].
- **Voltage scaling (V_{dd} reduction)**: reduces dynamic power quadratically. Used directly in [1] as part of the outer optimization loop over candidate V_{dd} values.

- **Flat iterative improvement** [2]: the closest prior work; solves allocation, assignment, module selection, scheduling, and clock/ V_{dd} selection simultaneously for *flat* DFGs. The seed paper extends this to hierarchical DFGs.

Key Tradeoffs

Table 1: Main Design Tradeoffs in Dataflow Optimization for HLS

Tradeoff	One side	Other side
Speed vs. quality	Flat synthesis: high quality	Hierarchical: fast but lower quality
Area vs. power	Resource sharing reduces area	Sharing increases switching activity $\rightarrow$ more power
Operation chaining	Saves clock cycles (fewer cycles needed)	Causes glitches $\rightarrow$ more power
Module reuse (Merge)	Saves area (fewer functional units)	Adds multiplexers $\rightarrow$ area/power overhead
V_{dd} reduction	Power scales as V_{dd}^2 : big savings	Slower gates $\rightarrow$ tighter timing constraint
Slack relaxation	Allows slower, cheaper modules	May reduce parallelism $\rightarrow$ longer schedule
Optimality vs. feasibility	ILP gives optimal solution	Exponential runtime; heuristic needed for large designs

Assumptions Behind Different Approaches

- **Seed paper** [1]:
 - Data-dominated behavior only (no control flow).
 - Fixed sampling period known at design time.
 - Fail-silent modules (no Byzantine faults).
 - Power model = switched-capacitance approximation.
- **Transformational approach** [3]:
 - Flat DFG only.
 - Algebraic equivalence of transformations must be verified; numerical stability not always preserved.
- **ILP-based methods**:
 - Exact optimum exists and is reachable.

- Runtime is acceptable only for small designs ($\lesssim 30$ operations in practice).
- **Simulated annealing [5]:**
 - Convergence to near-optimal solution given enough time.
 - Cooling schedule must be tuned per design; no automatic parameter selection.

Consequences for Embedded Systems Constraints

- **Timing / hard deadlines:**
 - The sampling period T_s is enforced as a hard constraint throughout synthesis.
 - Scheduling validity is checked after every move – no move is accepted that violates T_s .
 - Suitable for real-time DSP systems where deadline violations are not acceptable.
- **Energy / power:**
 - V_{dd} scaling + module selection + resource sharing together achieve up to $6.7\times$ power reduction.
 - Critical for battery-powered embedded devices (wearables, sensors, mobile).
- **Area / memory:**
 - RTL module merging reduces the number of functional units, which directly reduces chip area.
 - Smaller area $\rightarrow$ lower interconnect power (a secondary benefit not directly optimized but observed in experiments).
- **Determinism:**
 - The TDMA-style fixed schedule produced by HLS is fully deterministic – execution time of each operation is known statically.
 - This is ideal for safety-critical embedded systems (automotive, avionics) that require worst-case execution time (WCET) analysis.
- **Deployment constraints:**
 - The method targets ASICs; direct application to FPGAs requires adaptation (different resource model, LUT-based area estimation).
 - Control-flow intensive applications (e.g., embedded control software) cannot use this method directly.

Why Related Work is Sparse in Some Areas

- **Hierarchical power-optimized HLS** was a new problem in 1999; most prior work targeted either flat synthesis or area-only hierarchical synthesis. This explains the limited directly comparable work.

- **RTL module merging for non-isomorphic DFGs** had not been addressed before; the graph-embedding formulation and heuristic are original contributions with no direct predecessor.
- **Control-flow intensive HLS** is a neighboring problem class but requires fundamentally different techniques (e.g., control-data flow graphs, if-conversion, speculative execution); it is explicitly left as future work in [1].

References

- [1] G. Lakshminarayana and N. K. Jha, “High-Level Synthesis of Power-Optimized and Area-Optimized Circuits from Hierarchical Data-Flow Intensive Behaviors,” *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 18, no. 3, pp. 265–281, Mar. 1999.
- [2] A. Raghunathan and N. K. Jha, “An Iterative Improvement Algorithm for Low Power Datapath Synthesis,” in *Proc. Int. Conf. Comput.-Aided Des.*, Nov. 1995, pp. 597–602.
- [3] A. P. Chandrakasan, M. Potkonjak, R. Mehra, J. Rabaey, and R. W. Brodersen, “Optimizing Power Using Transformations,” *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 14, pp. 12–51, Jan. 1995.
- [4] P. G. Paulin and J. P. Knight, “Force-Directed Scheduling for the Behavioral Synthesis of ASICs,” *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 8, no. 6, pp. 661–679, June 1989.
- [5] S. Devadas and A. R. Newton, “Algorithms for Hardware Allocation in Data Path Synthesis,” *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 8, pp. 768–781, July 1989.